

Insertion Sort

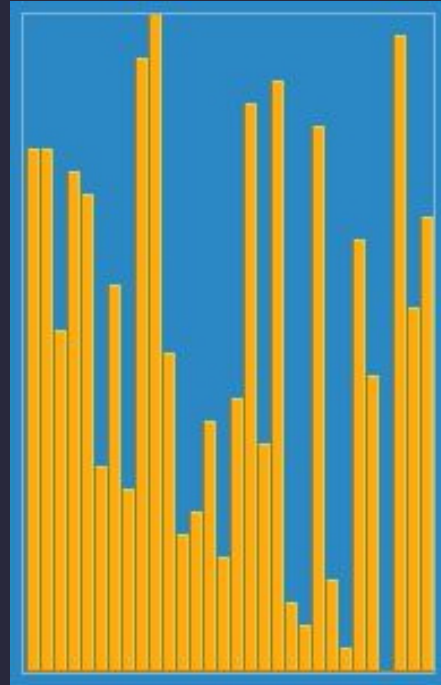
ALGORITMO DE ORDENAÇÃO

Resolução de Problemas
Estruturados em Computação



/CONCEITO GERAL

O insertion sort, ou ordenação por inserção, itera consumindo um elemento de entrada a cada repetição e aumenta uma lista de saída classificada. A cada iteração, a ordenação por inserção remove um elemento dos dados de entrada, encontra o local ao qual pertence na lista ordenada e o insere lá. Ele se repete até que nenhum elemento de entrada permaneça.

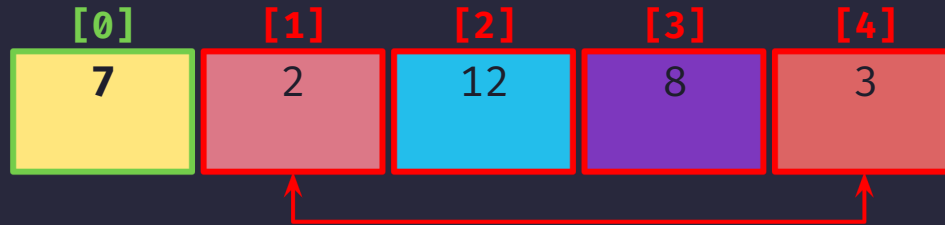


/ORDENANDO ELEMENTOS

[0]	[1]	[2]	[3]	[4]
7	2	12	8	3

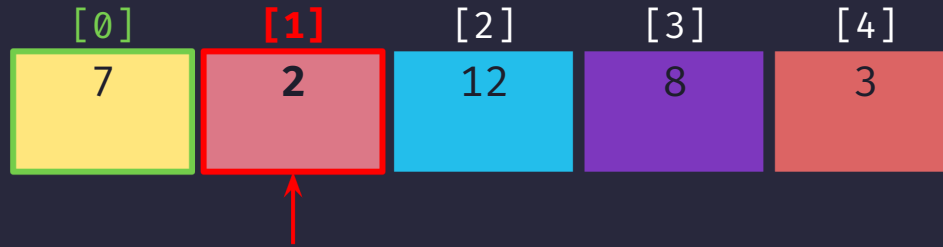
Vamos considerar parte do array como o array ordenado, e a outra parte será o array contendo os elementos de entrada (não-ordenados)

/ORDENANDO ELEMENTOS



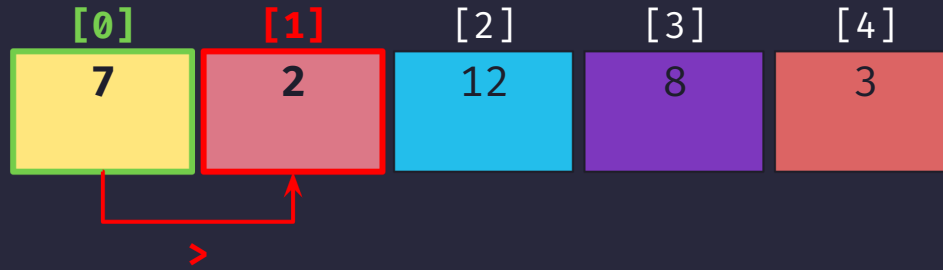
Assumimos que a posição [0] será
nosso array ordenado, e da posição
[1] a posição [4] nosso array com
elementos de entrada (não-ordenado)

/ORDENANDO ELEMENTOS



Vamos começar pegando o primeiro elemento do array de entrada ([1]).
Vamos “inserir” esse elemento no array ordenado.

/ORDENANDO ELEMENTOS



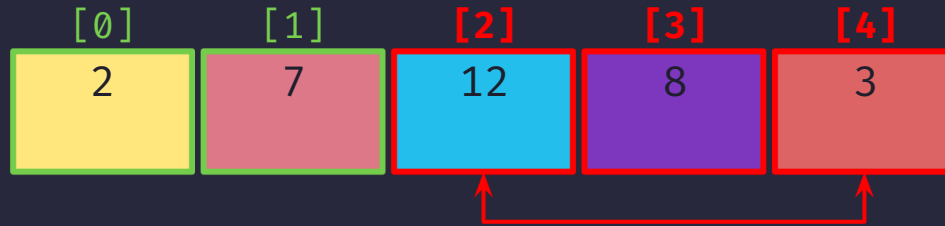
Verificamos se o elemento do array ordenado (atual - 1) é maior que o elemento que queremos inserir (atual = 1).

/ORDENANDO ELEMENTOS

[0]	[1]	[2]	[3]	[4]
2	7	12	8	3

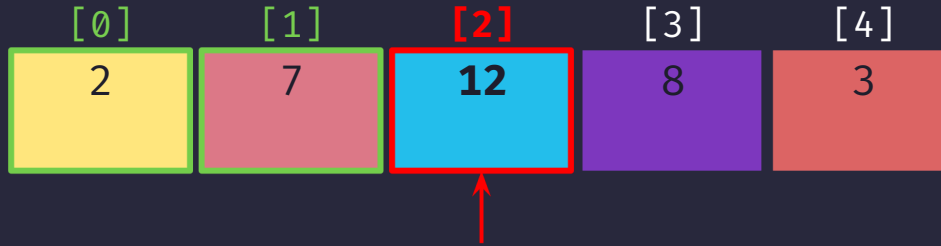
Como 7 é maior que 2, “inserimos” o 2 na posição do 7 e adicionamos o 7 à posição do 2 (swap).

/ORDENANDO ELEMENTOS



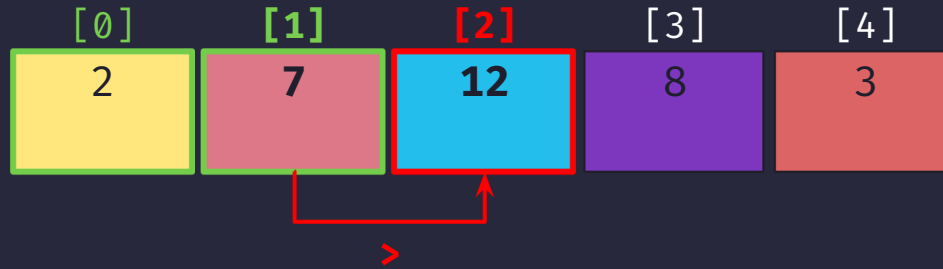
Agora o nosso array ordenado possui 2 posições, com valores já ordenados, e o nosso array de entrada possui 3 posições.

/ORDENANDO ELEMENTOS



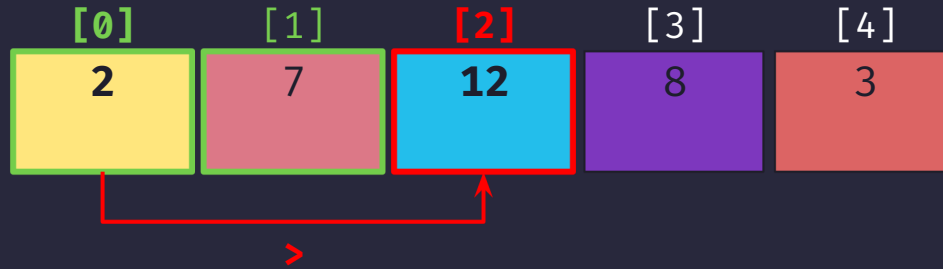
Agora vamos pegar o primeiro elemento do array de entrada ([2]). Vamos “inserir” esse elemento no array ordenado.

/ORDENANDO ELEMENTOS



Verificamos se o elemento anterior do array ordenado ($\text{atual} - 1$) é maior que o elemento que queremos inserir ($\text{atual} = 2$).

/ORDENANDO ELEMENTOS



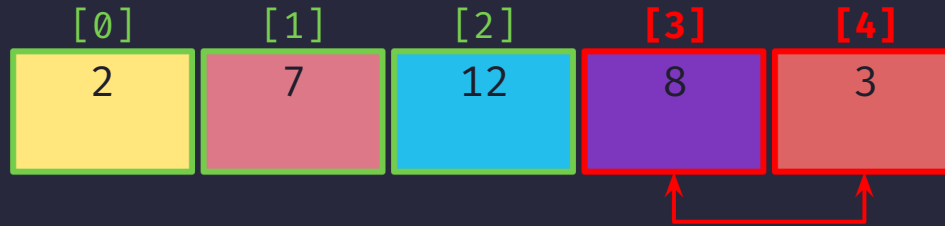
Verificamos então, se o primeiro elemento do array ordenado (atual - 2) é maior que o elemento que queremos inserir (atual = 2).

/ORDENANDO ELEMENTOS

[0]	[1]	[2]	[3]	[4]
2	7	12	8	3

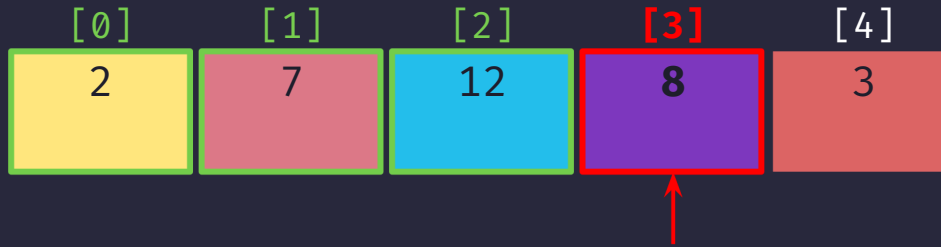
Como 7 e 2 não são maiores que 12, “inserimos” o 12 na última posição array ordenado (mantemos na mesma posição).

/ORDENANDO ELEMENTOS



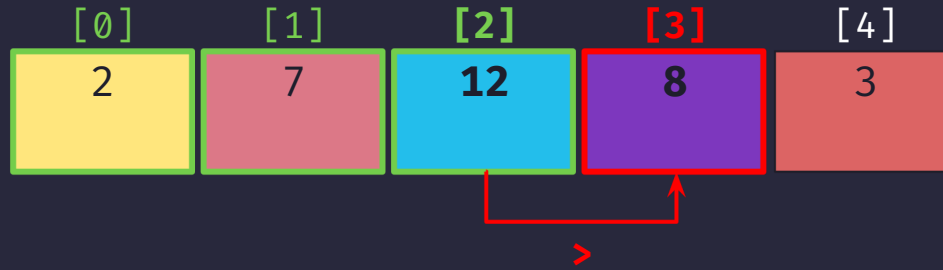
Agora o nosso array ordenado possui 3 posições, com valores já ordenados, e o nosso array de entrada possui 2 posições.

/ORDENANDO ELEMENTOS



Agora vamos pegar o primeiro elemento do array de entrada ([3]). Vamos “inserir” esse elemento no array ordenado.

/ORDENANDO ELEMENTOS



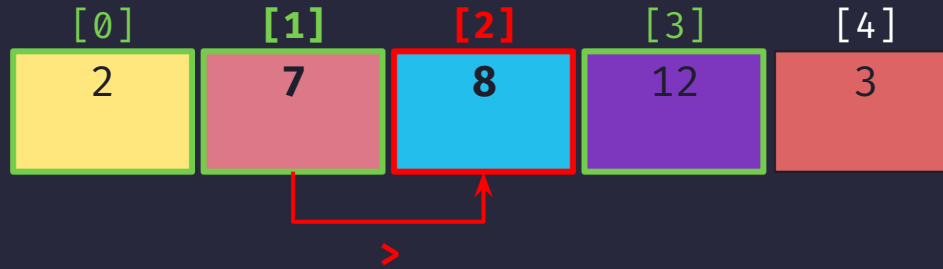
Verificamos se o elemento anterior do array ordenado (atual - 1) é maior que o elemento que queremos inserir (atual = 3).

/ORDENANDO ELEMENTOS



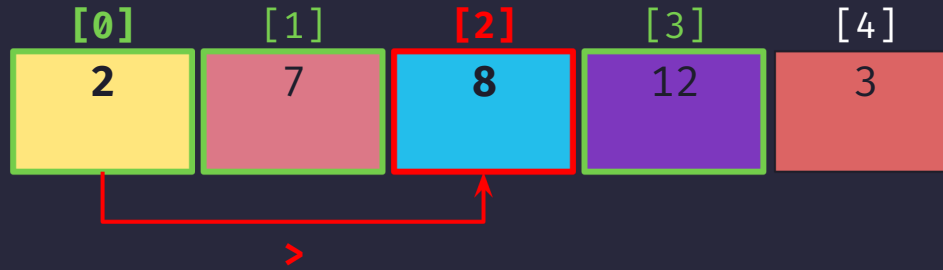
Como 12 é maior que 8, “inserimos” o 8 na posição do 12 e adicionamos o 12 à posição do 8 (swap).

/ORDENANDO ELEMENTOS



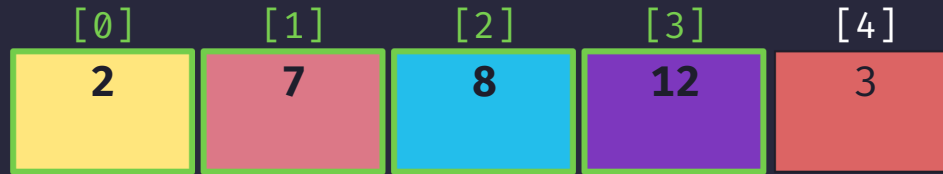
Verificamos então, se o elemento anterior ao atual (atual - 1) é maior que o elemento que estamos “inserindo” (atual = 2).

/ORDENANDO ELEMENTOS



Como 7 não é maior que 8, mantemos a mesma posição e verificamos com o primeiro elemento do array ordenado (atual - 2)

/ORDENANDO ELEMENTOS



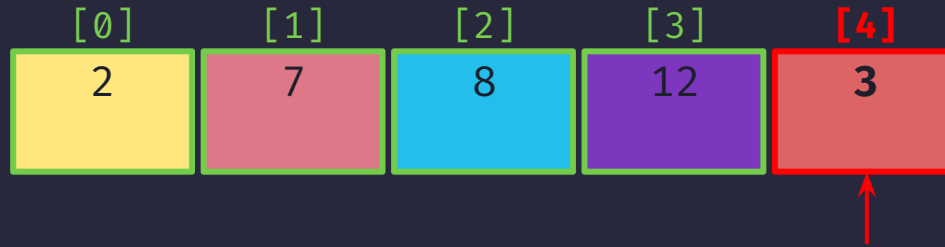
Como 2 também não é maior que 8, o 8 foi “inserido” na posição[2]do array ordenado

/ORDENANDO ELEMENTOS



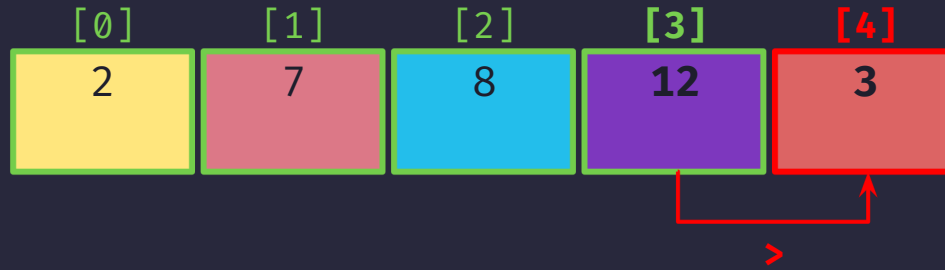
Agora o nosso array ordenado possui 4 posições, com valores já ordenados, e o nosso array de entrada possui apenas 1 posição.

/ORDENANDO ELEMENTOS



Agora vamos pegar o primeiro elemento do array de entrada ([4]). Vamos “inserir” esse elemento no array ordenado.

/ORDENANDO ELEMENTOS



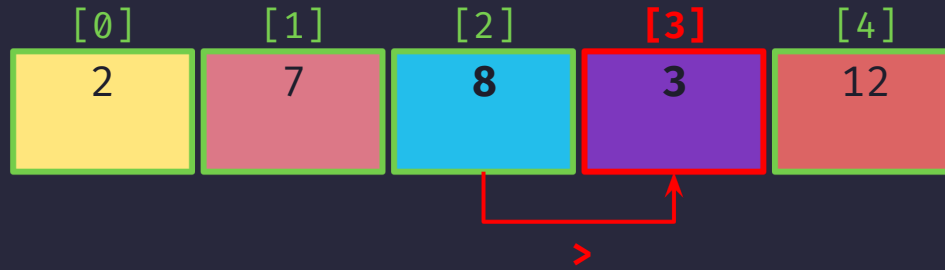
Verificamos se o elemento anterior do array ordenado (atual - 1) é maior que o elemento que queremos inserir (atual = 4).

/ORDENANDO ELEMENTOS



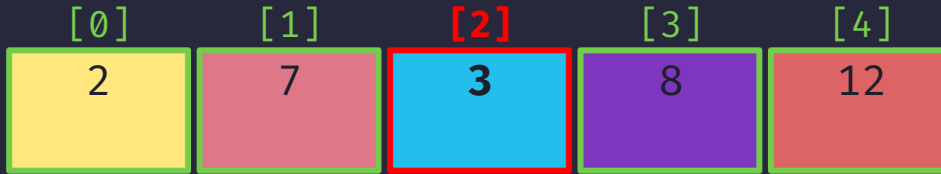
Como 12 é maior que 3, “inserimos” o 3 na posição do 12 e adicionamos o 12 à posição do 3 (swap).

/ORDENANDO ELEMENTOS



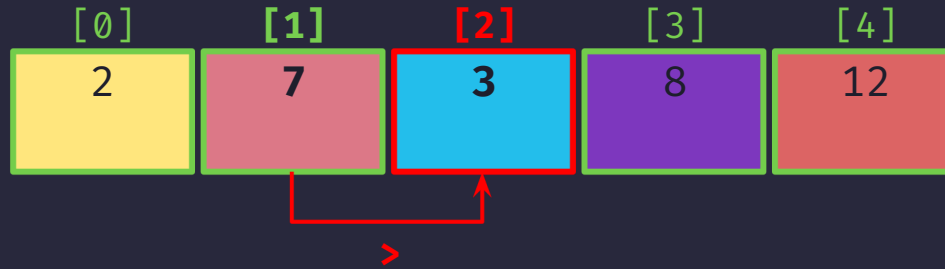
Verificamos então, se o elemento anterior ao atual (atual - 1) é maior que o elemento que estamos “inserindo” (atual = 3).

/ORDENANDO ELEMENTOS



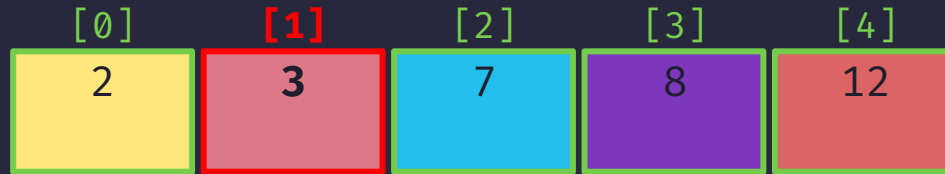
Como 8 é maior que 3, “inserimos” o 3 na posição do 8 e adicionamos o 8 à posição do 3 (swap).

/ORDENANDO ELEMENTOS



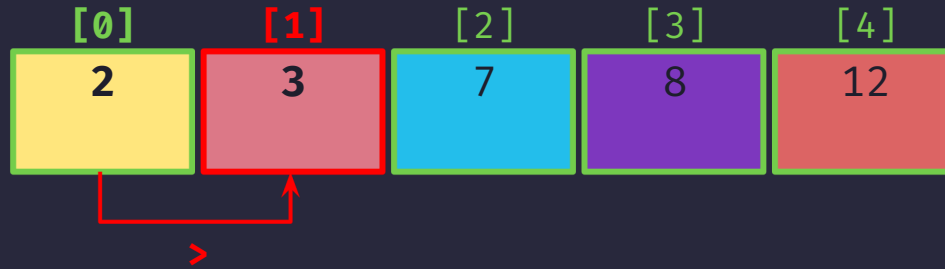
Verificamos então, se o elemento anterior ao atual (atual - 1) é maior que o elemento que estamos “inserindo” (atual = 2).

/ORDENANDO ELEMENTOS



Como 7 é maior que 3, “inserimos” o 3 na posição do 7 e adicionamos o 7 à posição do 3 (swap).

/ORDENANDO ELEMENTOS



Verificamos então, se o elemento anterior ao atual ($\text{atual} - 1$) é maior que o elemento que estamos “inserindo” ($\text{atual} = 1$).

/ORDENANDO ELEMENTOS

[0]	[1]	[2]	[3]	[4]
2	3	7	8	12

Como 2 não é maior que 3, o 3 foi “inserido” na posição [1] do array ordenado

/DICAS ÚTEIS

- Percebam que essa divisão dos arrays só acontece de forma imaginária, na realidade nunca dividimos o array. É só uma forma de interpretarmos para ficar mais fácil de associar o conceito.
- Diferente do bubble sort, no insertion sort quando pegamos um elemento, ele já é inserido na “posição correta” do array ordenado.
- No insertion sort não precisamos usar recursividade, visto que iremos ordenar um valor por vez e ao final da execução teremos a estrutura já ordenada, não é preciso que a função seja executada novamente.
- Utilize while ao invés de for.

